

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/384225481>

Fast Fourier Transform and its Applications

Article · December 2022

CITATIONS

0

READS

234

1 author:



[Anis Zebiane](#)

Notre Dame University – Louaize

4 PUBLICATIONS 0 CITATIONS

[SEE PROFILE](#)

Fast Fourier Transform and its Applications

Anis Bachir Zebiane

Notre Dame University - Louaize
Deir El Kamar

Department of Electrical, Computer and Communication Engineering
abzebiane@ndu.edu.lb

Abstract— This paper shows explanation of the theory of FFT, the difference between FFT and DFT, and visualize few applications of FFT in the real world. Then, we discussed one of these applications which is FFT in polynomial multiplication and showed all the related theories and explanation that verifies the relation between the two topics, and how FFT is used to reduce the complexity of solving such problem.

Keywords— Fourier, DFT, Signals, FFT, Inverse Fourier

I. INTRODUCTION

A signal can be considered as a physical variable that varies with time and have specific instruction or values. We can represent it in two ways: Continuous and Discrete. A signal, continuous or discrete, can be processed and analyzed using a method called Fourier Transform, in the frequency and time domain. Fourier Transform can be divided to 4 classes: Fourier Series, Discrete Fourier Transform, Discrete - time Fourier Transform and Fast Fourier Transform, where the latter will be discussed in this paper.

The Fast Fourier Transform is an advanced technology developed by Joseph Fourier in 1982, and re - studied later by 2 other scientists: James Cooley and John Tukey. FFT is a methodology that performs and computes the Discrete Fourier Transform (DFT) faster than any other method or algorithm.^[1] It converts any signal into spectral component and hence provide a frequency related information about the signal. Over a period of time, the signal will be sampled and distributed into frequency components.

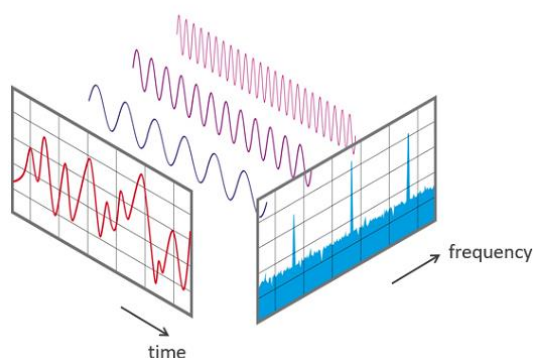


Figure 1: Signal in time and Frequency Domain

In Figure 1 we can see how the previously stated transformation is illustrated, and it is obvious how the signal is divided to 3 distinct frequencies.

II. DIFFERENCE BETWEEN FFT AND DTFT

Mathematically speaking, the Discrete - Time Fourier Transform (DTFT) is one type of the Fourier analysis that can be done to a sequence of values, that is usually used to divide a function to several samples and analyze them. It is considered to be continuous function of frequency, whereas the simple Discrete Fourier Transform (DFT) analyzes and studies the discrete samples.

Both methods are effective in the world of science and engineering, but here comes the importance of Fast Fourier Transform (FFT) technique. Basically, the difference between these techniques is that DTFT and DFT's work is being enhanced using FFT, where all of them uses Fourier series and transform but in different ways^[2]. The table below shows a brief idea about the difference between FFT and DTFT:

Table 1: Brief Difference between FFT and DTFT

	FFT	DTFT
Definition	The combination of multiple computing techniques including DFT and DTFT	Mathematical Technique that transforms time to frequency domains by dividing the signal into samples.
Work	Very Fast Analysis	Providing the relation between time and frequency domain
Applications	Solving partial differential equations, JPEG compression, etc....	Spectrum estimation, conviction, etc...
Version	Fast Version	Discrete Version

As we can see in Table 1, the difference between FFT and DTFT can be divided to many categories. The DTFT requires a big amount of time with a high number of samples N , whereas, FFT is an algorithm to make DTFT computations faster. The FFT hold one good assumption: that N is an integer multiple of two. This allows certain symmetries to occur, and hence reducing the number of calculations with less time. FFT has an advantage over DFT and DTFT because we have a decrease in the complexity from $O(N^2)$ to $O(N \log N)$ where N is representing the data points or number of samples. So, for example if $N = 1024 = 2^{10}$, $N \log N = 10240$.

Discrete Fourier Transform is used in processing digital systems applications such as calculating a signal's frequency spectrum, correlation analysis, computing polynomial multiplication, spectral analysis, and more. FFT has been used in other applications that include spectral analysis in analog video measurements, large integer and polynomial multiplication, filtering algorithms, and many other that will be well explained on the next section.

Summing up, we can say that the difference between the FFT and the traditional discrete time Fourier transform (DTFT) is that the DTFT is defined for a continuous interval (range) of frequencies, whereas the FFT is defined for a discrete set of frequencies. This means that the DTFT can be used to analyze the frequency content of a continuous-time signal, whereas the FFT is typically used to analyze the frequency content of a discrete-time signal.

III. FFT FOR DISCRETE SIGNALS

The steps to follow while doing FFT for discrete signals uses the strategy and formulas provided by DFT.

Assume we have a signal $\{x[n]\}_{n=0}^{N-1}$, then the signal is represented as a sequence $X[k]$ for $k = 0, 1, \dots, N-1$ with

$$(Eq. 1) \quad X[k] = \sum_{n=0}^{N-1} x[n] e^{-2\pi jnk/N}$$

We can transform this equation to the following:

$$(Eq. 2) \quad X[k] = \sum_{n=0}^{N-1} x[n] \omega_N^{nk}$$

The term ω_N^{nk} appearing in (Eq. 2) shows that there will be many calculations to happen on it by considering it a periodic function with N distinct values and is called the n th basis

vector with $\omega_n = \sum_{n=0}^{N-1} e^{-\frac{2\pi jn}{N}} (N: \text{#of samples})$.

For example, Taking $N=8$ (the FFT is simplest by far if is an integral power i.e. 2^3):

$$\omega_8^1 = e^{-j\frac{2\pi}{8}} = e^{-j45^\circ} = \frac{1-j}{\sqrt{2}} = a.$$

Then:

$$\begin{aligned} a^2 &= -j, & a^3 &= -ja = -a^*, & a^4 &= -1, \\ a^5 &= -a, & a^6 &= j \\ & \text{etc ...} \end{aligned}$$

So, we can now say:

$$\begin{aligned} \omega_8^4 &= -W_8^0 \\ \omega_8^5 &= -W_8^1 \\ \omega_8^6 &= -W_8^2 \\ \omega_8^7 &= -W_8^3 \end{aligned}$$

Based on the previous result, we can see that whenever the term nk gets outside the range 0-7, we will get the same results.

This idea can be expressed more formally as follows. Let X_0 and X_1 be the DFTs of 2 subsequences x_0 and x_1 each of length $N/2$, respectively. Then the DFT of x is given by the formula: $X[k] = \sum_{k=0}^{N/2-1} (X_0[k] \cdot \omega_n^{2k}) + \sum_{k=0}^{N/2-1} (X_1[k] \cdot \omega_n^{2k+1})$. We can now use this formula to derive the formula for the FFT. Let X' be the FFT of x , and let X'_0 and X'_1 be the FFTs of x_0 and x_1 , respectively.

Then we have:

$$X'[n] = \sum_{k=0}^{N/2-1} (X'_0[k] \cdot \omega_n^{2k}) + \sum_{k=0}^{N/2-1} (X'_1[k] \cdot \omega_n^{2k+1}).$$

We can then rearrange the terms in this equation to obtain the formula for the FFT:

$$X'[n] = \sum_{k=0}^{N/2-1} (X'_0[k] \cdot \omega_n^k) + \sum_{k=0}^{N/2-1} (X'_1[k] \cdot \omega_n^k \cdot \omega_n)$$

This is the formula for the FFT of a sequence x . It shows that the FFT of a sequence is a linear combination of the basis vectors, with the coefficients of the combination given by the elements of the original sequence. It also shows that the FFT of a sequence can be computed by recursively dividing the sequence into smaller subsequences and combining the results (Divide and Conquer approach). This is the key insight that allows the FFT to compute the DFT of a sequence with N elements in $O(N \log N)$ time, which is much faster than the $O(N^2)$ time required by the naive approach.

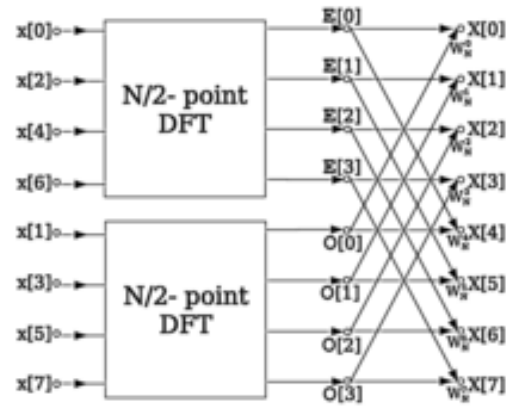


Figure 2: A typical FFT example that shows the decomposition into half size FFTs

IV. DIFFERENT APPLICATIONS OF FFT

As mentioned in the previous sections, FFT has a lot of applications. Mainly it is used in radar, sonar, audio processing, software used for radio and solving partial differential equations.

To clearly state the main applications of FFT, we can divide them into 2 parts:

1. Spectral Analysis
 - Removing low amplitude noise.
 - Filtering periodic signals and data.
 - Convolutions combined with FFT.
2. Image Processing
 - Blurring and Deblurring images.
 - Processing images that are matrices of RGB colors.
 - Red, green and blue (RGB) intensities.

For removing the noisy signals out of an audio, FFT is used to reconstruct the audio after dividing it and transforming it to a filtered spectrum with replacing the unwanted frequencies by zeros. Since we can differentiate the filters to 3 parts (low pass, high pass and band pass filters), FFT can do the filtering by transforming the signal to frequency domain, then remove the

elements of unwanted frequencies and finally re - obtain the filtered signal in time domain using iFFT (Inverse Fast Fourier Transform).

FFT Convolution involves the principle that a signal that we can get its response in time domain by convolution, can be used as multiplication with FFT in the frequency domain. So, for an input signal $x(t)$ with an impulse response $h(t)$, the response will be $y(t) = x(t) * h(t)$ (where $*$ represents the convolution sign). But, using FFT, we can easily transform the signal and its impulse response to the frequency domain, and the response will be directly calculated by: $Y(\omega) = X(\omega) H(\omega)$, where we can re - get our $y(t)$ by doing iFFT on $Y(\omega)$.

On the other hand, FFT can be used in blurring images by simply multiplying the image in frequency domain (that is obtained using FFT) with another image called Gaussian Image that is responsible of such operation [3]. It is used as well, to increase or decrease intensities of a color from a picture based on the RGB color set. Furthermore, FFT is used to get into a certain image and do certain operations on it like cropping it or even picking certain dimensions out of it.

V. FFT FOR POLYNOMIAL MULTIPLICATION [4]

Suppose we have 2 polynomials:

$$p(x) = a_0 + a_1x + \dots + a_{n-1}x^{n-1}$$

$$q(x) = b_0 + b_1x + \dots + b_{n-1}x^{n-1}$$

Then, we can write their multiplication as:

$$s(x) = p(x) \cdot q(x) = c_0 + c_1x + \dots + c_{2n-2}x^{2n-2}$$

$$\text{Where } c_i = \sum_{\max\{0, i-(n-1)\}}^{\min\{i, n-1\}} a_k b_{i-k}$$

To be more specific, let's assume we were given two polynomials $A(x)$ and $B(x)$, and we were supposed to calculate their multiplication (product) $C(x) = A(x) \cdot B(x)$. Solving such task, will cost a running time of approximately $O(n^2)$ using regular mathematics techniques, where any polynomial is represented as:

$$A(x) = \sum_{j=0}^{n-1} a_j x^j \text{ with } a = a_0, a_1, \dots, a_{n-1}.$$

Since most of the people don't like to dig inside mathematics problems and theorems, we should ask ourselves if there is a simpler and easier way to solve such problem. The answer is yes. The way to do this, is by taking the polynomial all itself and represent it in point - value form, and then we can easily calculate the product. In order to do this, we extract a polynomial of degree n . Let's name it $A(x)$, that is represented as:

$$\sum_{i=0}^{n-1} (x_i, y_i) = \{(x_0, y_0), (x_1, y_1), \dots, (x_{n-1}, y_{n-1})\} \text{ where all } y_i = A(x_i) \text{ for } i=0, 1, 2, \dots, n-1.$$

Taking an example of $A(x) = x^3 - 4x + 1$. Here, the point - value representation is as follows: $\{(0,1), (1, -2), (2,1), (3,16)\}$. We can see that the time complexity of calculating $A(x)$ for all the n different points is $O(n^2)$.

Now, we can see that after transforming our task to point - value visualization, it is definitely easy to compute our main requirement which is: $C(x) = A(x) \cdot B(x)$, which will now take a complexity of $O(n)$. $C(x)$ here will be having a degree

of $2n$, which means to calculate $2n$ values of y , we need $2n$ values of x .

Using a phenomenon called Interpolation, we can recover our answer but in vector notation. In order to do so, we use the following formula: $\sum_{k=0}^{n-1} y_k \cdot \frac{\prod_{j \neq k} (x - x_j)}{\prod_{j \neq k} (x_k - x_j)}$.

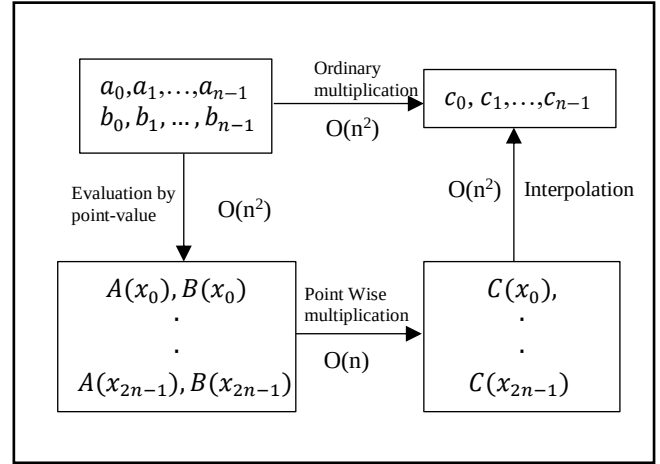


Figure 3: Representation of regular polynomial multiplication

Figure 3 briefly shows what we discussed so far.

As we can see, this problem is still taking $O(n^2)$ to be solved. We can easily move to a time complexity of $O(n \log n)$ by choosing the points very carefully. So, if we picked the evaluation points to be the "complex roots of unity", we can reach the point - value representation by using discrete Fourier Transform (DFT) for a coefficient vector, then the interpolation of the coefficient vector can be reached by performing the inverse discrete Fourier Transform (iDFT). Here comes the importance of our famous Fast Fourier Transform (FFT) which will do both operations (DFT and iDFT) in a time complexity of $O(n \log n)$.

FFT follows the method of divide and conquer to complete this problem. Let's introduce 2 new polynomials of degree $n/2$ that divides $A(x)$ to even and odd parts as follows:

$$A_0(x) = a_0 + a_2(x) + a_4(x) + \dots + a_{n-2}x^{\frac{n}{2}-1}$$

$$A_1(x) = a_1 + a_3(x) + a_5(x) + \dots + a_{n-1}x^{\frac{n}{2}-1}$$

And now, we can simplify our $A(x)$ expression by taking some common factors and summations to be:

$$A(x) = A_0(x^2) + xA_1(x^2)$$

So, instead of evaluating $A(x)$ at $\omega_n^0, \omega_n^1, \omega_n^2, \dots, \omega_n^{n-1}$ using the discrete Fourier Transform method (where ω_n^k represents the complex root of unity), we can easily simplify our work by evaluating our $A_0(x)$ and $A_1(x)$ of degree $n/2$ at the new evaluation points that are: $(\omega_n^0)^2, (\omega_n^1)^2, (\omega_n^2)^2, \dots, (\omega_n^{n-1})^2$

It is obvious that this list of points have instead of n distinct values, there is $n/2$ complex $n/2$ th roots of unity. And we can notice that both polynomials $A_0(x)$ and $A_1(x)$ are evaluated in a recursive way at the n complex n th roots of unity. Calculating now the total complexity time of this recurrence, we have: $T(n) = 2T(n/2) + O(n) = O(n \log n)$.

Everything we previously stated can be simplified in the following divide and conquer picture:

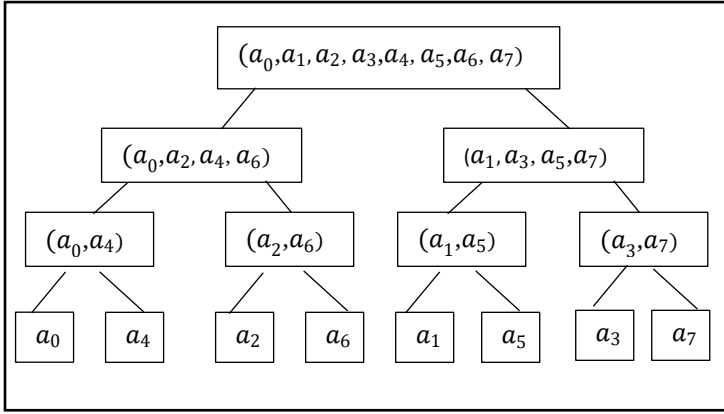


Figure 4: Representation of divide and conquer strategy for the given polynomial

Now, because of the following properties:

- $\omega_{n/2}^k = \omega_n^{2k}$
- $\omega_n^{2k+n} = \omega_n^{2k}$
- $\omega_n^{k+\frac{n}{2}} = -\omega_n^k$

We can write that:

$$y_k = y_k^0 + \omega_n^k y_k^1 = A^0(\omega_n^{2k}) + \omega_n^k A^1(\omega_n^{2k}) = A(\omega_n^k)$$

$$y_{k+\frac{n}{2}} = y_k^0 - \omega_n^k y_k^1 = A^0(\omega_n^{2k}) + \omega_n^{k+\frac{n}{2}} A^1(\omega_n^{2k})$$

$$= A^0(\omega_n^{2k+n})$$

Thus, using recursive FFT the vector y is re-reached, where y can be as well the DFT of the input vector a .

To sum up, FFT successfully showed how a certain polynomial multiplication that is done by transforming into point-value form, can be done in a fast and more familiar way than DFT and regular mathematics multiplication.

VI. CONCLUSIONS

As a conclusion, we can say that clearly FFT has more benefits than DTFT and DFT. In this paper, we found out how DFT has to perform $O(n^2)$ multiplication and addition for a big number of points, whereas FFT can directly take the input point and produce the desired output by using some adding and multiplication process that takes around $O(n \log n)$ as a time complexity. Implemented polynomial multiplication showed how FFT have solved the task in less time which reduces calculations and turns the systems to be more efficient and applicable. The previously stated details, allows us to acknowledge that using FFT and other improved technologies, all the future work and problems will decrease the complexity of operations such as multiplication to make any system more efficient.

ACKNOWLEDGMENT

I would like to thank Dr. Nadine Abou Dargham, for her continuous support and for supporting my ideas by her outstanding experience in Fourier and FFT during our EEN 340 course.

REFERENCES

- [1] https://en.wikipedia.org/wiki/Fast_Fourier_transform.
- [2] H. Magsi, A. H. Sodro, Fast Fourier Transform by Verilog. Sukkur Institute of Business Administration. Sukkur, Pakistan. November 2020.
- [3] ZettaBytes, EPFL. 2017, Mar 30. Three Applications of the (Fast) Fourier Transform (ft. Michael Kapralov). YouTube. <https://www.youtube.com/watch?v=aqa6vyGSdos>
- [4] Y. Jia. Polynomial Multiplication and Fast Fourier Transform. Com S 477/577 Notes. 2020, Sep 17.